Ders 14 Genişletilmiş Notu: Seminer Sunumları: IoT Verisinde Ardışık Veri Gruplama, Metin Sıkıştırma Karşılaştırması ve CAS-CNN

Ders 14 Genişletilmiş Notu: Seminer Sunumları: IoT Verisinde Ardışık Veri Gruplama, Metin Sıkıştırma Karşılaştırması ve CAS-CNN

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yapısı
- 2. Sunum 1: Ardışık Veri Gruplamaya Dayalı IoT Veri Sıkıştırma
- 3. Sunum 2: Metin Sıkıştırma Yöntemlerinin Karşılaştırılması
- 4. Ara Konu: Aritmetik Kodlamada Hangi Sayı Gönderilir?
- 5. Sunum 3: CAS-CNN ile Görüntü Sıkıştırma Yapaylıklarının Bastırılması
- Örnek Sorular

1. Dersin Yapısı

Bu ders, dönemin son seminer haftasıdır. Önceki derslerde olduğu gibi her sunumda bir öğrenci veri sıkıştırmayla ilgili yayımlanmış bir makaleyi ekran paylaşarak anlattı; hoca sunum sırasında ve sonunda sorular sordu, anlaşılmayan yerleri sınıfla birlikte çözmeye çalıştı.

Üç sunum yapıldı. İlki nesnelerin interneti (IoT) sensörlerinde enerji tasarrufu için basit, kayıplı bir sıkıştırma yöntemiydi. İkincisi dört metin sıkıştırma yöntemini (RLE, aritmetik kodlama, Punctured Elias kodu, Goldbach kodu) küçük veriler üzerinde karşılaştıran bir makaleydi. Üçüncüsü JPEG sıkıştırmasının yarattığı yapaylıkları derin evrişimli bir sinir ağıyla gideren CAS-CNN çalışmasıydı. İkinci ve üçüncü sunum arasında bir öğrenci, aritmetik kodlamada son aralıktan hangi sayının gönderileceğini sordu.

2. Sunum 1: Ardışık Veri Gruplamaya Dayalı IoT Veri Sıkıştırma

2.1 Motivasyon: Sensörlerde Enerji

Bir öğrenci “Internet of Things Data Compression Based on Successive Data Grouping” başlıklı makaleyi sundu (Türkçesi: ardışık veri gruplamasına dayalı nesnelerin interneti veri sıkıştırması).

Nesnelerin interneti akıllı şehirlerden sağlığa ve otomotive kadar pek çok alanda kullanılan, büyük miktarda veri üreten bir teknolojidir; bu veriler işlenir, analiz edilir ve karar destek sistemlerinde kullanılır. Sunan öğrenci sensörleri “nesnelerin internetinin sinir uçları” diye tanımladı. Sensörlerin çoğu pille çalışır ve biten pilin değiştirilmesi hem maliyetli hem de işletme açısından zordur. Bu yüzden sensörlerde enerji tasarrufu önemlidir.

Makalenin çıkış noktası şu gözlemdir: sensörlerin enerji tüketiminin ortalama %80’i, topladıkları verinin baz istasyonuna ya da sunucuya gönderilmesi sırasında harcanır. Gönderilen veri

azaltılırsa enerji de azalır. Dolayısıyla sensörde veri sıkıştırma doğrudan bir enerji tasarrufu yöntemidir.

2.2 Karşılaştırılan Mevcut Yöntemler

Sunumda sensör verisinde yaygın kullanılan üç yöntem kısaca tanıtıldı, çünkü sonuçlar bunlarla karşılaştırılıyor:

- **Compressive Sensing (CS):** Sensör verisini daha az ölçümle temsil ederek sıkıştırır. Sıkıştırma algoritması karmaşık olduğu için bellek ihtiyacı ve enerji tüketimi yüksektir.
- **Standard RBM:** Derin öğrenme tabanlı bir yöntemdir. Eğitim sunucu tarafında yapılır; sensörde az parametreli basit bir ağ çalışır. Ağır işler sunucuda olduğu için sensörün harcadığı enerji ve kapasite çok yüksek değildir.
- **Stacked RBM-AE:** Birden fazla katman kullanarak derin öğrenmeyi işe daha çok katan ve sıkıştırmayı iyileştiren yöntemdir.

2.3 Önerilen Yöntem

Önerilen yöntemde sıkıştırma oranı önceden sabitlenmez, verinin kendisine göre oluşur. Yalnız basit işlemler kullanıldığı için sensörde kapasite ve enerji daha az harcanır.

Yöntemin dayanağı sensör verisinin doğasıdır: sensörler çoğunlukla ardışık değerler üretir ve ardışık değerler arasındaki farklar çoğu zaman ihmal edilebilir düzeydedir. Bu kadar küçük bir farkın iletilmesinin sunucu tarafında ve veri yönetiminde pek anlamı yoksa, bu değerler gruplanır ve tek bir veri gibi gönderilir.

"Ne kadar fark ihmal edilebilir?" sorusunun cevabı sunucudaki eğitim verisinden hesaplanan bir eşiktir: RMSSD (root mean square of successive differences, ardışık farkların karesel ortalamasının karekökü). Slayttaki formül (ekrandan, 00:09:45):

$$RMSSD = \sqrt{\frac{1}{N-1} \times \sum_{i=1}^{N} (v_i - v_{i-1})^2}$$

Slayttaki sözde kod (ekrandan, 00:09:45) şöyledir:

```
Algorithm 1 Calculate RMSSD
1: Input TrainingSensedValues dataset;
2: Diff <- 0;
3: RMSSD <- 0;
4: for i <- 1 to TrainingSensedValues.length do
5:     Diff <- Diff + Absolute(TrainingSensedValues[i] - TrainingSensedValues[i-1]);
6:     RMSSD <- Diff / (TrainingSensedValues.length - 1);
7: end for
8: return RMSSD;
```

> **Ders sonrası not:** Sözde kod farkların karesini alıp karekök hesaplamıyor, mutlak farkların ortalamasını hesaplıyor; yani formülle uyuşmuyor. Ayrıca formülde toplam $i = 1$'den başlıyor; v_0 tanımlı olmadığı için büyük olasılıkla $i = 2$'den başlaması gerekir. Sunumda bu fark konuşulmadı.

Karar kuralı (ekrandan, 00:09:45):

$$\text{Decision} = \begin{cases} \text{group with previous} & \text{if } |v_i - v_{i-1}| \leq RMSSD \\ \text{store as new value} & \text{otherwise} \end{cases}$$

Sıkıştırma algoritması bu kurala göre verileri tek tek alır, öncekiyle farkına bakar: fark küçükse değeri gruba katar ve grubun sayaç değerini artırır, büyükse yeni bir kayıt açar. Gönderilen her kayıt bir değer ve o değerin kaç kez tekrar ettiğini gösteren bir sayıdan (occurrence) oluşur.

2.4 Sıkıştırılmış Verinin Açılması

Sunucu tarafında açma algoritması (ekrandan, 00:10:50) her kayıttaki değeri, occurrence sayısı kadar sonuç listesine ekler:

```
Algorithm 3 Reconstruct Data
1: Input CompressedValues dataset;
2: Result <- Empty List with one column (value);
3: for i <- 0 to CompressedValues.length do
4:     for j <- 0 to CompressedValues[i].occurrence do
5:         Result.Add(CompressedValues[i]);
6:     end for
7: end for
8: return Result;
```

Bu yüzden yöntem kayıplıdır: gruptaki küçük farklar geri gelmez. Makalede kaybın oranının küçük olacağı öngörülüyor.

Hoca sunum bitince veriyi azaltan algoritmanın ne olduğunu ayrıca sordu, çünkü RMSSD'nin bir başarı ölçütü mü yoksa sıkıştırmanın parçası mı olduğu ilk anlatımda net değildi. Öğrenci bir örnekle açıkladı:

Soru: Veriyi azaltan algoritma tam olarak ne yapıyor, açarken orijinale nasıl ulaşılıyor? **Cevap:** Elimizde 100 ardışık değer olsun. İlk 30'u birbirinden RMSSD'den küçük farklarla değişiyorsa (biri 23,005, sonraki 23,004 gibi), bu 30 değer tek bir değer olarak gönderilir ve yanına "30 tane" bilgisi eklenir. Açarken aynı değer art arda 30 kez yazılır. Orijinal değerlere ulaşılamaz, bu yüzden yöntem kayıplıdır.

Hoca bunu görüntü sıkıştırmada renklerin azaltılmasına benzetti: veri kayıplıdır, ama örneğin 50 değer sıkıştırılıp açıldığında yine 50 değer elde edilmelidir; yalnız değerlerin bazıları birbirinin aynısı olur.

2.5 Deney ve Değerlendirme Ölçütleri

Uygulama Intel Lab kablosuz sensör ağı verisiyle yapıldı (ekrandan, 00:11:55). Bu veri setinde sıcaklık, nem, ışık ve voltaj değerleri vardır; örnek olarak sıcaklık verisi kullanıldı. Sonuçlar CS, Standard RBM ve Stacked RBM-AE'nin aynı veri üzerindeki sonuçlarıyla karşılaştırıldı. Üç ölçüt kullanıldı:

- **Compression ratio (CR):** Orijinal veri boyutunun sıkıştırılmış veri boyutuna bölümü; verinin kaçta birine düştüğünü gösterir.
- **Percentage of root-mean-square difference (PRD):** Çıktı verisi ile orijinal veri arasındaki farkın yüzde olarak ifadesi; yani kayıp.
- **Quality Score (QS):** CR'ın PRD'ye oranı, $QS = CR/PRD$. Hem yüksek sıkıştırmayı hem düşük kaybı birlikte ödüllendirir.

2.6 Sonuçlar

Sonuç grafiklerinde (ekrandan, 00:12:45) karşılaştırılan üç yöntemin CR değeri 10'da sabit tutulmuş; önerilen yöntemde CR yaklaşık 11 çıkmış. PRD grafiğinde CS yaklaşık 38, Standard RBM yaklaşık 33, Stacked RBM-AE yaklaşık 10 düzeyinde; önerilen yöntemin çubuğu sıfıra çok yakın. QS logaritmik ölçekte gösterilmiş: diğerleri 10 ile 100 arasında, önerilen yöntem 10^6 civarında.

2.7 Enerji Tüketimi

Makalenin asıl amacı enerji olduğu için tüketim, belli formüllerle ağ türüne göre hesaplanmış. Tablodaki bileşenler (ekrandan, 00:14:25): α_u veri gönderme (mW/Mbps), α_d veri alma (mW/Mbps), β ise işlem sırasında her durumda harcanan sabit enerji (mW).

Ağ türü	α_u (mW/Mbps)	α_d (mW/Mbps)	β (mW)
Wi-Fi	283,17	137,01	132,86

Ağ türü	α_u (mW/Mbps)	α_d (mW/Mbps)	β (mW)
3G	868,98	122,12	817,88
LTE	438,39	51,97	1288,04

Ortalama tüketimin karşılaştırması (ekrandan, 00:14:25):

Ağ türü	Orijinal veri (mW)	Önerilen yöntem (mW)
Wi-Fi	509,9	184,48
3G	1974,98	976,29
LTE	1871,78	1367,95

Wi-Fi'de tüketim yaklaşık üçte birine iner ($184{,}48/509{,}9 \approx 0{,}36$). 3G'de yaklaşık yarıya ($\approx 0{,}49$), LTE'de yaklaşık %27 azalır ($\approx 0{,}73$). Sunan öğrencinin açıklaması: 3G ve LTE'de sabit bileşen β daha büyük olduğu için, gönderilen veri azalsa da oransal tasarruf daha düşüktür. Yine de her ağ türünde önerilen yöntem tasarruf sağlıyor.

3. Sunum 2: Metin Sıkıştırma Yöntemlerinin Karşılaştırılması

3.1 Amaç ve Deney Düzeni

İkinci sunumda bir öğrenci, metin verisinde dört sıkıştırma algoritmasını karşılaştıran bir makaleyi anlattı: Run-Length Encoding, aritmetik kodlama, Punctured Elias kodu ve Goldbach kodu. Hoca bu konunun derste görülen konulara yakın, görece hafif bir makale olduğunu belirtti; Elias kodu derste geçmişti, Goldbach kodunu ise ilk kez duyduğunu söyledi.

Giriş kısmında veri sıkıştırmanın saklama alanını ve internet üzerinden veri gönderme süresini azaltmak için sürekli çalışılan bir alan olduğu anlatıldı. 1990'dan 2020'ye internet kullanıcı sayısı hızla artmış; 2019'da dünya nüfusunun yarısından fazlası internete en az bir kez erişmiş. Ne kadar az veri gönderilirse ağ trafiği o kadar rahatlar.

Makalede 40, 80 ve 120 bitlik (5, 10 ve 15 baytlık) üç veri dört algoritmayla sıkıştırılmış (ekrandan, 00:23:35). Sonuç tablosuna geçmeden önce her algoritma kısaca anlatıldı.

3.2 Run-Length Encoding

RLE veriyi run'lara, yani (sayaç, değer) ikililerine dönüştürür. Sunumdaki örnek (ekrandan, 00:24:10) kalp şekilli bir piksel görüntüsüdür; arka plan pikseli `x`, kalp pikseli `y` ile gösterilmiş. Satırların RLE karşılıkları:

```
xxxxxxxxx   -> 9x
xxyyxyyxx   -> 2x 2y 1x 2y 2x
xyyyyyyyx   -> 1x 7y 1x
xyyyyyyyx   -> 1x 7y 1x
xxyyyyyxx   -> 2x 5y 2x
xxxyyyxxx   -> 3x 3y 3x
xxxxyxxxx   -> 4x 1y 4x
xxxxxxxxx   -> 9x
```

Slayttaki hesap: görüntü 9×9 piksel, her piksel 1 bayt sayılırsa 81 bayt. Toplam 22 run vardır; her run'da değer ve sayaç 1'er bayt olduğundan sıkıştırılmış hali $22 \times 2 = 44$ bayttır. Yukarıdaki sekiz satırın run sayıları $1 + 5 + 3 + 3 + 3 + 3 + 3 + 1 = 22$ eder (ara adım eklendi).

3.3 Aritmetik Kodlama

Aritmetik kodlamada matematik daha fazladır. Önce metindeki karakterlerin olasılıkları bulunur, sonra [0,1) aralığı bu olasılıklarla orantılı parçalara bölünür. İlk karakterin aralığı alınır, genişletilir ve aynı oranlarla yeniden bölünür; sonraki karakterde de aynısı yapılır. Son karakterden sonra elde edilen aralıktaki sayı mesajın sıkıştırılmış halidir; bu sayı ve olasılıklarla mesaj geri izlenerek çözülebilir.

Sunumdaki örnek “HEY!!” metnidir (ekrandan, 00:25:10): $P(H) = 1/5$, $P(E) = 1/5$, $P(Y) = 1/5$, $P(!) = 2/5$. [0,1) aralığı H için [0; 0,2), E için [0,2; 0,4), Y için [0,4; 0,6), ! için [0,6; 1) olarak bölünür. H kodlanınca aralık [0; 0,2) olur ve bu aralık aynı oranlarla H [0; 0,04), E [0,04; 0,08), Y [0,08; 0,12), ! [0,12; 0,2) diye bölünür. Sonraki adımlar slaytta animasyonla gösterildi; sonuç aralığı bu bölümün “Örnek Sorular” kısmında hesaplandı.

3.4 Elias Gamma ve Punctured Elias Kodu

Elias gamma kodu bir önek (prefix) kullanan, derste görülmüş bir koddur. Slaytta 1’den 20’ye kadar Elias gamma kodları listelenmiş (ekrandan, 00:25:58); ilk değerler:

```
1 -> 1        2 -> 010       3 -> 011       4 -> 00100
5 -> 00101    6 -> 00110     7 -> 00111     8 -> 0001000
16 -> 000010000
```

Punctured Elias kodunda ise sayının ikili hali ters yazılır ve önekler kısaltılır. Bu yüzden bazı değerler Elias koduna göre daha kısa, bazıları daha uzun çıkar. Slayttaki karşılaştırma tablosunun (Table 7, Comparison of Punctured and Elias codes) ilk satırları (ekrandan, 00:25:58):

Değer	P1	P2	Elias	biased Elias	avantaj P1	avantaj P2
0	0	01	—	1	0	-1
1	101	001	1	010	0	0
2	1001	1011	010	011	-1	-1
3	11011	0001	011	00100	0	1
4	10001	10101	00100	00101	0	0
7	1110111	00001	00111	0001000	0	2
8	100001	101001	0001000	0001001	1	1
15	111101111	000001	0001111	000010000	0	3
16	1000001	1010001	000010000	000010001	2	2

Slayttaki bir notta, değer 127’den büyükse ortalama sıkıştırmanın 3,15’ten 3,10 bit/bayta indiği yazıyor (ekrandan, 00:25:58). Sunan öğrenci bu sonucun Punctured Elias kodunun asıl makalesinde verildiğini ve kodun çok verimli olduğunun söylenemeyeceğini belirtti.

> **Ek bilgi (derste anlatılmadı):** Punctured Elias kodları büyük olasılıkla P. Fenwick’in tamsayı kodları üzerine çalışmalarından gelir. Bu kodlar, sayıların ikili gösterimindeki 1 bitlerinin konumlarını işaretleyerek uzunluğu sayının kendisine değil bit örüntüsüne bağlar; bu yüzden avantaj sütununda bazı değerlerde kazanç, bazılarında kayıp görülür.

3.5 Goldbach Kodu

Goldbach kodu, Goldbach sanısından (conjecture) türetilmiş bir algoritmadır: “2’den büyük her pozitif çift sayı iki asal sayının toplamıdır.” Slayttaki adımlar (ekrandan, 00:27:20):

1. Metindeki bütün karakterler okunur ve her karakterin frekansı hesaplanır. En sık görülen karakter birinci sıraya ($n = 1$) konur, ötekiler buna göre sıralanır.
2. Her karakter için bir kod sözcüğü bulunur: $2(n + 3)$ çift sayısını veren iki asal bulunur. Asallar 3, 5, 7, … diye başlar.
3. Kod sözcüğünde yalnız iki tane “1” vardır; asal araması ikinci “1”de durur, yani kod sözcüğü her zaman 1 ile biter.

“GOOD MORNING” örneği (ekrandan, 00:30:00). Sıkıştırmadan önce her karakter 8 bitlik ASCII ile yazılır:

Karakter	Frekans	ASCII	İkili	Bit	Bit × Frekans
G	2	71	01000111	8	16
O	3	79	01001111	8	24
D	1	68	01000100	8	8

Karakter	Frekans	ASCII	İkili	Bit	Bit × Frekans
Boşluk	1	83 (slaytta)	01010011	8	8
M	1	77	01001101	8	8
R	1	82	01010010	8	8
N	2	78	01001110	8	16
I	1	73	01001001	8	8
Toplam					96

> **Ders sonrası not:** Boşluk karakterinin ASCII kodu 32'dir (ikili 00100000); slayttaki 83 ve 01010011 'S' harfine aittir. Her karakter 8 bit sayıldığı için toplam 96 bit değişmez.

Sıkıştırmadan sonra:

Karakter	Frekans	n	$2(n+3)$	Asallar	Kod sözcüğü	Bit	Bit × Frekans
O	3	1	8	3 + 5	11	2	6
G	2	2	10	3 + 7	101	3	6
N	2	3	12	5 + 7	011	3	6
D	1	4	14	3 + 11	1001	4	4
Boşluk	1	5	16	5 + 11	0101	4	4
M	1	6	18	7 + 11	0011	4	4
R	1	7	20	7 + 13	00101	5	5
I	1	8	22	5 + 17	010001	6	6
Toplam							41

Sonuçta metin 96 bitten 41 bite iner; yaklaşık yarı yarıya bir küçülme ($96/41 \approx 2{,}34$).

Burada hoca "Primes" sütununun kod sözcüğünü oluştururken nasıl kullanıldığını sordu. Sunan öğrenci kuralı tam açıklayamadı; hoca sınıfla birlikte tabloya bakarak çözdü (aşağıdaki "Örnek Sorular" bölümünde ayrıntısı var). Kural şudur: asallar küçükten büyüğe 3, 5, 7, 11, 13, 17, … diye sıralanır ve her asal kod sözcüğünde bir bit konumuna karşılık gelir. Toplamda kullanılan iki asalın konumuna 1, diğer konumlara 0 yazılır ve kod ikinci 1'de biter. Örneğin $8 = 3 + 5$ için ilk iki konum 1'dir: `11`. $12 = 5 + 7$ için 3 yok (0), 5 var (1), 7 var (1): `011`. $22 = 5 + 17$ için 3 yok, 5 var, 7, 11, 13 yok, 17 var: `010001`.

Hoca bu kuralın nereden geldiğini de vurguladı: tabloya bir "Primes" sütunu konduysa bu bilginin bir yerde kullanılması gerekir, yoksa o sütun konmazdı.

Her kod sözcüğünde tam iki 1 bulunduğu ve kod ikinci 1 ile bittiği için hiçbir kod sözcüğü bir başkasının öneki olamaz; kod çözücü bit akışını okurken ikinci 1'i gördüğü anda bir sembolün bittiğini anlar.

> **Ek bilgi (derste anlatılmadı):** Bazı çift sayıların birden fazla asal ayrışımı vardır; örneğin $10 = 3+7 = 5+5$, $22 = 3+19 = 5+17 = 11+11$. Tabloda hangi ayrışımın seçildiği derste açıklanmadı. Seçilen ayrışımlar tablodaki kod uzunluklarını belirler; büyük olasılıkla makalede ayrışım seçimi için ayrı bir kural tanımlanmıştır.

3.6 Deney Sonuçları

Dört yöntemle elde edilen çıktı boyutları (ekrandan, 00:31:58):

Deney	Girdi	RLE	Aritmetik	Punctured Elias	Goldbach
#01	40 bit (5 bayt)	80 bit	32 bit	64 bit	14 bit
#02	80 bit (10 bayt)	64 bit	56 bit	75 bit	28 bit
#03	120 bit (15 bayt)	240 bit	65 bit	88 bit	49 bit

Goldbach kodu her deneyde en küçük çıktıyı verdi; ikinci sırada aritmetik kodlama geldi. Punctured Elias ve RLE iyi sonuç vermedi. RLE birinci deneyde veriyi iki katına, üçüncü deneyde

iki katına çıkardı (40 → 80, 120 → 240); Punctured Elias birinci deneyde de veriyi büyüttü. Sıkıştırma oranı grafiklerinde (ekrandan, 00:32:40) de RLE hep altta, Goldbach genelde üstte kaldı.

Sonuç bölümünde, internette resim, video ve dosya boyutları büyüdükçe sıkıştırma algoritmalarının hem depolama hem de trafik için önem kazandığı söylendi. Sunan öğrenciye göre RLE yeni yöntemlere göre eski kalmıştır, ama sıkıştırılacak verinin türüne göre seçilirse daha iyi kullanılabilir.

3.7 Hocanın Deney Verisi Hakkındaki Soruları

Hoca üç deneyin verisinin içeriğini sordu: metin mi, sayı mı? Çünkü verinin türü RLE'nin başarısını doğrudan etkiler. Sunan öğrenci makalede verinin içeriğinin verilmediğini, yalnız 40, 80 ve 120 bit olarak tanımlandığını gösterdi. Hoca makalenin başlığında "text" geçtiğine dikkat çekti: metin işleme denince genelde karakterlerden oluşan veri anlaşılır. Metinde aynı karakterin art arda gelmesi azdır, bu yüzden RLE'nin metinde başarısız olması beklenir.

> **Hocanın vurgusu:** Bir yöntemin başarısını ölçüyorsanız ve karşılaştırmada RLE varsa, RLE'nin başarısız olacağı bilinen bir veri içeriğini tek başına almamalısınız. Makalenin farklı veri türleriyle daha fazla deney yapması gerekirdi.

Makale 2017 tarihli, bir uluslararası mühendislik ve teknoloji dergisinde yayımlanmış. Hoca bunun çok eski bir çalışma olmadığını not etti.

4. Ara Konu: Aritmetik Kodlamada Hangi Sayı Gönderilir?

İkinci sunumdan sonra bir öğrenci aritmetik kodlamayla ilgili bir soru sordu. Öğrenci, son aralıkta low ile high arasındaki en kısa bitle yazılabilen değerin alınması gerektiğini hatırlıyordu, ama bir soru çözümünde low değer üzerinden hesap yapılmıştı.

Soru: Aritmetik kodlamada son aralıktan low değeri mi almalıyız, yoksa aradaki daha az bitle yazılabilen bir değeri alabilir miyiz? (Öğrencinin örneğinde low 0,825, high 0,85 idi.) **Cevap:** Low ile high arasındaki herhangi bir sayı kodlamak için alınabilir. Ama high alınamaz, çünkü aralık üstten açıktır: alt uç köşeli parantezle (kapalı), üst uç normal parantezle (açık) yazılır, yani $[low, high)$. Genelde low kullanılır, çünkü aralık örneğin 3 ile 5 arasındaysa küçük olanın bit sayısı büyük olanınkinden ya da eşittir. Kitaplarda ve kod yazarken hep low alınır.

Öğrenci, sayı değeri küçük olsa bile noktadan sonraki basamak sayısının bit sayısını belirlediğini, yani 0,825 yerine aradaki daha kısa bir değerin daha az bitle yazılabileceğini söyledi. Hoca bunun üzerine aritmetik kodlamada sayının nasıl ele alındığını hatırlattı:

- Aritmetik kodlamada metin 0 ile 1 arasında bir sayıya sıkıştırılır. Sayının float olduğu zaten bilindiği için hep noktanın sağ tarafıyla işlem yapılır.
- Derste float sayılarla çarpma ve bölme zor olduğu için ilk dört basamağı tamsayı gibi alıp kaydırarak çalışan bir sürüm gösterilmişti. Yazılmış aritmetik kodlayıcıların büyük çoğunluğu bu yöntemi kullanır.
- Sayıyı float olarak düşünmemek gerekir; noktadan sonraki kısmın tamsayı karşılığının bit gösterimi kullanılır. Örneğin 0,8125 için aslında 8125'in bit karşılığı gönderilir; açarken bunun "0." ile başladığı bilinir.

Hoca sorunun asıl çözümüne o an erişemediği için çözümü sonradan öğrenciye göndereceğini söyledi.

> **Ders sonrası not:** Hocanın "küçük olanın bit sayısı her zaman daha azdır ya da eşittir" sözü, basamakları tamsayı olarak kodlama bakış açısında doğrudur. Sayı ikili kesir olarak gönderildiğinde ise aralıktaki en kısa ikili kesir büyük olasılıkla low'un kendisi değil, aralığın içindeki bir değer olabilir; öğrencinin sorusu bu duruma yönelikti. Her iki yaklaşımda da seçilen değer $[low, high)$ içinde kalmalıdır.

5. Sunum 3: CAS-CNN ile Görüntü Sıkıştırma Yapaylıklarının Bastırılması

5.1 Problem

Son sunumda bir öğrenci “CAS-CNN: A Deep Convolutional Neural Network for Image Compression Artifact Suppression” makalesini anlattı (ekrandan, 00:51:50). Görüntü sıkıştırmada ortaya çıkan yapaylıklar (artifact) için bir yöntem önerilmektedir.

Kayıplı görüntü sıkıştırma, web üzerinden iletilen ve depolanan görüntülerin boyutunu azaltmak için yaygın kullanılır. Ancak yüksek sıkıştırma oranları kullanıcı tarafında görsel yapaylıklara, bozulmalara yol açar. Kayıpsız yöntemlerde görsel bir sorun yoktur; kayıplı yöntemler ise geri çevrilemez. Özellikle yüksek oranlarda, sıkıştırılmış ve orijinal görüntü arasındaki farklar sıkıştırma şemasına özgü yapaylıklar olarak görünür hale gelir. Bu yapaylıklar yalnız göze hoş görünmemekle kalmaz, birçok düşük seviyeli görme algoritmasını da olumsuz etkiler.

Birçok sıkıştırma algoritması görüntüyü bloklara ayırır, her bloğa bir dönüşüm uygular, sonra nicemleme ve genel bir kayıpsız sıkıştırma yapar. JPEG bu yaklaşımın en yaygın standardıdır: pek çok açık kaynaklı kodlayıcısı vardır, birçok kamerada donanım hızlandırıcı olarak bulunur. JPEG görüntüler günlük hayatta, tarihi belgelerde ve yasal soruşturmalarda kanıt olarak sunulan kayıtlarda yaygındır.

Görüntü sıkıştırma kablosuz sensör sistemlerinde de görsel bilginin sensör düğümlerinden merkezi depolama ve işleme birimine aktarılması için kullanılır. Bu düğümler pille çalışır ve en pahalı iş verinin iletilmesidir; iletim enerjisini azaltmak için güçlü sıkıştırma yapılır ve bunun bedeli yapaylıklardır. Benzer bir durum veri depolayan mobil cihazlarda da vardır. Buna karşılık alıcı tarafta enerji kısıtı azdır: görüntüyü açmak ve sonradan işlemek için çok daha fazla hesaplama gücü vardır. Yapaylık bastırma bu yüzden alıcı tarafta yapılabilir. (Bu, ilk sunumdaki “enerjinin çoğu iletimde harcanır” gözlemiyle aynı motivasyondur.)

Derin evrişimli sinir ağları görüntü sınıflandırma, nesne algılama ve anlamsal bölütleme gibi görevlerde insan performansını aşarak bilgisayarla görmenin önemli bir aracı haline gelmiştir; düşük seviyeli görüntü ve video işleme, regresyon görevlerinde de önem kazanmıştır. Makalenin katkıları: yapaylıkları gidermek için 12 katmanlı bir evrişimli ağ mimarisi, bu ağı eğitmek için bir strateji ve LIVE1 veri setinde kapsamlı bir değerlendirme.

5.2 Önceki Yöntemler

Sıkıştırma yapaylıklarını bastıran yöntemler arasında sunumda şunlar geçti: ayrık kosinüs dönüşümüne dayalı SA-DCT, BM3D (Block Matching and 3D Filtering) ve DRCN gibi ağlar. Bu ağların sonuçları başka makalelerden alınıp karşılaştırmada kullanılmış.

Asıl karşılaştırılan yöntem AR-CNN’dir (Artifact Reduction Convolutional Neural Network), çünkü doğrudan bu yapaylıkları gidermek için eğitilmiştir. AR-CNN özellikle JPEG sıkıştırma sonrası oluşan bloklaşma ve kenar çevresindeki bozulmaları hedefler. Dört katmanlı ağdan daha derin, beş katmanlı bir ağ kullanır; farklı JPEG kalite faktörleri ve veri kümeleri için eğitilen ağlar arasında transfer öğrenme uygular. AR-CNN’in veri setinde 500 görüntü vardır: 400 eğitim, 100 test; ayrıca Twitter’dan seçilen görüntülerle ikinci bir eğitim yapılmış.

5.3 Ağ Mimarisi

Mimari şekli (ekrandan, 01:00:00) dört aşamadan oluşur. Ana yol şöyledir:

```
Ana yol (soldan sağa):
input (1 x h x w) -> A (128 x h x w)
  -> DS -> B (128 x h/2 x w/2)
  -> DS -> C (256 x h/4 x w/4)
  -> DS -> D (256 x h/8 x w/8)

Geri dönüş (sağdan sola), her aşamada:
  alt aşamanın sonucu US ile büyütülür ve concat ile
  aynı çözünürlükteki özelliklerle birleştirilir
  (Stage 1'de birleşim boyutu 257 x h x w)
```

```
Çok ölçekli çıktılar: B^ (1 x h/2 x w/2), C^ (1 x h/4 x w/4), D^ (1 x h/8 x w/8)
Son çıktı: A^ -> output (1 x h x w)

Kayıp: son çıktı için MSE loss; ara çıktılar için eşit ağırlıklı çok ölçekli MSE loss
```

Şekildeki renkler: ana yol (kalın mavi), alt seviye özelliklerin birleştirilmesi (kırmızı), çok ölçekli çıktı yolları (yeşil) ve çok ölçekli çıktıların yeniden kullanımı (turuncu). A'dan D'ye kadar her blok iki evrişim katmanından oluşur. Ağ derinleştikçe kanal sayısı 1'den 128'e, sonra 256'ya çıkar. Çözünürlük her aşamada 2×2 alt örneklemeyle (downsampling, DS) yarıya iner; sunan öğrenci bunu ortalama havuzlama (average pooling) katmanı olarak açıkladı. Böylece 12 katmanlı bir evrişimli ağ elde edilir.

Sunuma göre ağ önce standart eğitim yöntemleriyle son teknoloji doğruluğa ulaşacak şekilde eğitilir. Ortalama yol uzunluğunu kısaltmak için her aşamadaki yüksek çözünürlüklü ara sonuçlar, bir alt aşamadan üst örneklenmiş (US) sonuçlarla birleştirilip bir sonraki katmanda kullanılır. Bundan iki fayda beklenir: yüksek çözünürlüklü ayrıntıların korunması ve ara katmanları atlayan bağlantılarla ağın ilk katmanlarının eğitiminin kolaylaşması.

5.4 Performans Ölçütleri

Slayttaki ölçütler (ekrandan, 01:01:50):

- **MSE (ortalama karesel hata)** ve onunla yakından ilişkili **PSNR (tepe sinyal-gürültü oranı)**, görüntüler ve diğer sinyaller arasındaki farkı değerlendirmede en yaygın ölçütlerdir. MSE, bozulmuş ve referans görüntü arasındaki yoğunluk farkının karesinin piksel bazında ortalamasıdır. PSNR desibel cinsinden ifade edilir ve mümkün olan maksimum sinyal değerine göre normalize edilmiş MSE'dir.
- İki ölçüt de tamamen referanslıdır: pikselleri tek tek orijinal görüntüyle karşılaştırır ve mükemmel bir yeniden yapılandırmada MSE sıfıra yaklaşır. Algılanan görsel kaliteden farklı oldukları bilinir, ama basit oldukları için yaygın kullanılırlar.
- **IPSNR**, PSNR'ın başlangıçtaki bozuk görüntüye göre artışıdır; dolayısıyla kalite gelişimini ölçer ve farklı veri kümeleri arasında daha kararlıdır.
- **SSIM (yapısal benzerlik endeksi)** popüler bir alternatiftir; parlaklık, kontrast ve yapı benzerliğini değerlendiren üç terimin çarpımının ortalamasıdır.

  Ders sonrası not: Sunan öğrenci ölçütleri anlatırken "basitlikleri nedeniyle yaygın kullanım alanı bulunmamaktadır" dedi; slaytta doğru biçimi "yaygın kullanım alanı bulunmaktadır" diye yazıyor (ekrandan, 01:01:50).

  Ek bilgi (derste anlatılmadı): 8 bitlik görüntülerde PSNR genellikle $PSNR = 10\log_{10}\left(\frac{255^2}{MSE}\right)$ dB olarak hesaplanır; MSE küçüldükçe PSNR büyür. Tablodaki PSNR-B, büyük olasılıkla bloklaşma etkisini ayrıca cezalandıran PSNR çeşididir.

5.5 Eğitim ve Kayıp Fonksiyonu

Kayıp fonksiyonu grafiği (ekrandan, 01:03:08) eğitimi iki faza ayırır: önce çok ölçekli kayıpla, sonra yalnız çıktı kaybıyla (ince ayar) eğitilir. Grafikte yeşil ve kırmızı eğriler test setindeki çıktı kaybı, sarı ve mavi eğriler eğitim setindeki kayıptır; sarı eğri çok ölçekli kaybı gösterir ve okunabilirlik için 3 kat büyütülmüştür. Geçiş yaklaşık 200. epoktadır. İnce ayar aşamasında bir epok 50 000 yerine 150 000 görüntü içerir.

Veri: AR-CNN LIVE1 kullanmıştı; bu çalışmada eğitim için kamuya açık, renkli görüntülerden oluşan büyük ImageNet 2013 veri seti kullanıldı. Sunumda 396 bin eğitim ve 20 bin doğrulama görüntüsü olarak ayrıldığı söylendi. Görüntülerden 120×120 piksellik kesitler alındı, renk uzayı dönüştürüldü ve yalnız parlaklık kanalı kullanıldı. Ağın girdisi bu tek kanallı görüntülerin JPEG ile sıkıştırılmasıyla oluşturuldu. Görüntülere ağa verilmeden önce herhangi bir ön işlem uygulanmadı.

Ağ önce kalite faktörü 20 ile sıkıştırılmış girdi için geliştirildi; eğitim süresini kısaltmak için diğer kalite faktörlerinin ağları bundan başlatılarak eğitildi. Eğitimde Titan X (Maxwell) GPU kullanıldı; ileri geçiş ölçümü GTX 1080 üzerinde tek hassasiyetli kayan noktayla, 1,1 megapikselli görüntüde yapıldı.

5.6 Sonuçlar

LIVE1 üzerindeki karşılaştırma tablosu (Table II, Restoration Quality Comparison on LIVE1; ekrandan, 01:05:15):

QF	Algoritma	PSNR (dB)	PSNR-B (dB)	SSIM
10	JPEG	27,77	25,33	0,791
10	SA-DCT	28,65	28,01	0,809
10	AR-CNN	29,13	28,74	0,823
10	L4	29,08	28,71	0,824
10	ours, MS loss	29,36	28,92	0,830
10	ours, w/ loss FT	29,44	29,19	0,833
20	JPEG	30,07	27,57	0,868
20	SA-DCT	30,81	29,82	0,878
20	AR-CNN	31,40	30,69	0,890
20	L4	31,42	30,83	0,890
20	L8	31,51	30,92	0,891
20	ours, MS loss	31,67	30,84	0,894
20	ours, w/ loss FT	31,70	30,88	0,895
40	JPEG	32,35	29,96	0,917
40	SA-DCT	32,99	31,79	0,924
40	AR-CNN	33,63	33,12	0,931
40	L4	33,77	—	—
40	ours, MS loss	33,98	32,83	0,935
40	ours, w/ loss FT	34,10	33,68	0,937
60	JPEG	33,99	31,89	0,940
60	ours, w/ loss FT	35,78	35,10	0,954
80	JPEG	36,88	35,47	0,964
80	ours, w/ loss FT	3?,55	37,73	0,973

Her kalite faktöründe önerilen ağın (ince ayarlı, “w/ loss FT”) PSNR’ı en yüksek çıktı. Kalite faktörü (QF) JPEG’in sıkıştırma ayarıdır; QF küçüldükçe sıkıştırma artar, yapaylık çoğalır ve ağın sağladığı kazanç büyür. Örneğin QF 10’da JPEG’e göre yaklaşık 1,67 dB, AR-CNN’e göre 0,31 dB artış var (ara adım eklendi: $29{,}44 - 27{,}77$ ve $29{,}44 - 29{,}13$). Hoca tablodaki QP sütununu sordu; sunan öğrenci bunun kalite faktörü olduğunu söyledi. Sunumda veri setinden örnek görüntüler de gösterildi (yapaylıkların belirgin olduğu görüntüler).

5.7 Hocanın Değerlendirmesi

Soru: Makale standart CNN üzerinde tam olarak ne değiştirmiş? Gösterilen mimariden bu anlaşılmıyor. **Cevap:** Sunan öğrenci, makalede derin sinir ağını eğitmek için farklı bir strateji uygulandığının söylendiğini, ama bu kısmın açılmadığını belirtti. Katmanlar arasında aktivasyonla ilgili bir değişiklik yapıldığını, ancak bu kısmı tam anlamadığı için açıklayamadığını söyledi.

Hoca, CNN mimarisini bilmeden bu tür bir makaledeki ayrıntıları anlamanın zor olduğunu, değişikliğin yalnız parametre ya da gizli katman sayısıyla oynamak olmadığını düşündüğünü belirtti. Mimari şeklinden okunabilen fark, çok ölçekli çıktılar, çok ölçekli kayıp ve alt seviye özelliklerin üst seviyeyle birleştirilmesidir.

Örnek Sorular

Soru 1: Goldbach kodunda asallar kod sözcüğünü nasıl belirler?

(ekrandan, 00:30:00; derste sözlü, 00:28:45)

“GOOD MORNING” tablosunda her karakterin n sırası, $2(n+3)$ değeri ve bu değeri veren iki asal yazılı. Örneğin O için $8 = 3 + 5$, kod `11`; N için $12 = 5 + 7$, kod `011`; I için $22 = 5 + 17$, kod `010001`. “Primes” sütunu kod sözcüğünü oluştururken nasıl kullanılıyor?

Çözüm (derste verildi): Asallar küçükten büyüğe sıralanır: 3, 5, 7, 11, 13, 17, ... Her asal kod sözcüğünde bir bit konumudur. Toplamı oluşturan iki asalın konumuna 1, aradaki asalların konumuna 0 yazılır; kod ikinci 1'de biter.

- $8 = 3 + 5$: 3 var (1), 5 var (1) → `11`.
- $10 = 3 + 7$: 3 var (1), 5 yok (0), 7 var (1) → `101`.
- $12 = 5 + 7$: 3 yok (0), 5 var (1), 7 var (1) → `011`.
- $14 = 3 + 11$: 1, 0, 0, 1 → `1001`.
- $22 = 5 + 17$: 3 yok, 5 var, 7 yok, 11 yok, 13 yok, 17 var → `010001`.

Toplam uzunluk: $3 \cdot 2 + 2 \cdot 3 + 2 \cdot 3 + 4 + 4 + 4 + 5 + 6 = 41$ bit; sıkıştırmadan önce $12 \times 8 = 96$ bit (python ile doğrulandı).

Soru 2: "HEY!!" metninin aritmetik kodlaması

(ekrandan, 00:25:10)

$P(H) = 1/5$, $P(E) = 1/5$, $P(Y) = 1/5$, $P(!) = 2/5$ ve aralık sırası H, E, Y, ! iken "HEY!!" metninin son aralığını bulun.

> **Çözüm (derste verilmedi):** Başlangıç [0, 1); alt aralıklar H [0; 0,2), E [0,2; 0,4), Y [0,4; 0,6), ! [0,6; 1). H: [0; 0,2). E: genişlik 0,2, aralık [0,04; 0,08). Y: genişlik 0,04, aralık [0,056; 0,064). !: genişlik 0,008, aralık [0,0608; 0,064). !: genişlik 0,0032, aralık [0,06272; 0,064). Son aralık $[0{,}06272;\ 0{,}064)$; genişliği $0{,}00128 = (1/5)^3(2/5)^2$. Bu aralıktaki herhangi bir sayı (genelde low) mesajı temsil eder. Bilgi içeriği $-\log_2 0{,}00128 \approx 9{,}61$ bittir (python ile doğrulandı).

Soru 3: RLE ile kalp görüntüsünün boyutu

(ekrandan, 00:24:10)

Her pikseli 1 bayt olan 9×9'luk kalp görüntüsü satır satır RLE ile kodlanıyor; her run'da değer ve sayaç 1'er bayt. Sıkıştırılmış boyut nedir?

Çözüm (derste verildi): Satırların run sayıları 1, 5, 3, 3, 3, 3, 3, 1 olmak üzere toplam 22 run vardır. Her run 2 bayt olduğundan $22 \times 2 = 44$ bayt; orijinal 81 bayttır.

Soru 4: IoT gruplama yönteminde veri nasıl azaltılır ve nasıl açılır?

(derste sözlü, 00:15:27)

Çözüm (derste verildi): Ardışık iki değerin farkı RMSSD eşiğinden küçük ya da ona eşitse değer öncekiyle gruplanır; gruplar (değer, tekrar sayısı) olarak gönderilir. Örneğin 100 değerin ilk 30'u birbirine çok yakınsa, tek değer ve "30" gönderilir. Açarken değer 30 kez yazılır. Değer sayısı korunur ama gruptaki küçük farklar kaybolur; yöntem kayıplıdır.

Soru 5: Aritmetik kodlamada son aralıktan hangi sayı gönderilir?

(derste sözlü, 00:37:05)

Son aralık low = 0,825 ve high = 0,85 olarak bulunmuşsa hangi sayı gönderilmelidir?

Çözüm (derste verildi): $[low, high)$ aralığındaki herhangi bir sayı gönderilebilir; high aralığa dahil olmadığı için gönderilemez. Kitaplarda ve kodlarda genelde low alınır. Sayının "0." ile başladığı bilindiği için yalnız noktadan sonraki kısım kodlanır.

Dersin Toparlaması

Bu son seminer haftasında üç makale incelendi. IoT sunumu, enerjinin çoğunun iletimde harcandığı sensörlerde basit bir kayıplı yöntemin (ardışık farkı RMSSD eşiğinin altında kalan değerleri gruplayıp değer ve tekrar sayısı olarak göndermek) sıkıştırma oranını, kaybı ve enerji tüketimini karmaşık yöntemlerden daha iyi dengeleyebildiğini gösterdi. Metin karşılaştırması derste görülen RLE ve aritmetik kodlamayı Elias ailesinden bir kod ve Goldbach sanısına dayanan bir önek koduyla yan yana koydu. Hoca burada yöntemin başarısının deney verisinin

türüne bağlı olduğunu, tekrarı az olan metinde RLE’nin kötü çıkmasının beklendiğini vurguladı.

CAS-CNN sunumu kayıplı sıkıştırmanın diğer tarafına baktı: JPEG yapaylıklarını alıcı tarafta, çok ölçekli bir evrişimli ağla bastırmak ve kaliteyi PSNR, PSNR-B ve SSIM ile ölçmek. Ara konu olarak aritmetik kodlamada son aralığın üstten açık olduğu, aralıktaki herhangi bir sayının gönderilebileceği, uygulamada genelde low değerin noktadan sonraki basamaklarıyla kodlandığı hatırlatıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.